

The Return of Magic: Causal Compression in Adaptive Systems

Leo Guinan

Draft — September 2026

Abstract

A passenger presses a button, a model produces an image, or a traveler crosses a continent. The local invocation is small; the consequence depends on a much larger historical arrangement of knowledge, computation, infrastructure, coordination, and maintenance. This paper develops causal compression as a framework for that asymmetry. Causal compression converts historically expensive sequences into maintained transitions that are cheap to invoke. We distinguish representational causal compression, in which history is reduced to a future-sufficient internal state, from structural causal compression, in which historical work is embodied in maintained infrastructure that makes a transition available at low local cost. Adaptive systems frequently couple the two. Compression expands reachable states and lowers invocation burden, but selection also converts possibilities into commitments whose maintenance can consume future adaptive headroom. The phenomenology of magic is therefore located in the invocation–construction gap: a reliable, consequential transition appears magical when the work required to construct its reliability is much larger than the work required to invoke it locally, while its causal depth is hidden from the invocation loop. We present a lifecycle model, a worked software example, and a bounded experimental probe of queue migration. In synthetic interventions, the proposed instrumentation recovers deliberately constrained lifecycle stages and identifies evaluation as the first downstream bottleneck when generation pressure is increased. A trace-derived workload replay tests robustness to a non-synthetic arrival process. These results validate the instrument under declared models; they do not establish general organizational or AI-system effects. The paper is a conceptual framework rather than a completed formal theory; its equations are definitions, accounting identities, or model-relative hypotheses unless separately proved or measured.

1 Introduction: the disappearing cause

A passenger presses a button, receives a generated image, crosses a continent, or asks a system to summarize a difficult argument. The local action is small. The consequence is not. Between them lies a causal chain of computation, accumulated design, trained expertise, contracts, energy, maintenance, institutional coordination, and physical infrastructure. Most of that chain is absent from the user’s immediate field of action.

This is not necessarily deception. It is often the point of a successful system. A mechanism becomes useful when a previously expensive sequence can be invoked through a smaller interface. The work has not vanished; it has been displaced in time, distributed across actors, and compiled into the present structure of the world. A technical system can therefore feel like magic without supernatural causation and without being a trustworthy black box.

We call the relevant mechanism *causal compression*. The term has prior uses in causal time-series analysis [1] and in work connecting compression with learned causal structure [16]. Our object is related but distinct: the compression of a causal pathway into a reusable transition in an adaptive

system. The compressed object may be a learned policy, a software library, an airport system, a laboratory protocol, or an institutional procedure.

The central claim is deliberately modest:

A transition can appear magical when the causal work required to make it reliable is much larger than the work required to invoke it locally.

This claim has three parts: the transition must be reachable, reliable enough for the declared contract, and supported by historical work that is largely absent from the local invocation loop. The relevant combination is a large construction–invocation gap, reliable operation, substantial consequence, and hidden causal depth.

The paper’s mechanism is:

causal compression $\rightarrow$ cheap invocation $\rightarrow$ expanded reachability $\rightarrow$ selection
 $\rightarrow$ commitment $\rightarrow$ maintenance burden $\rightarrow$ headroom loss
 $\rightarrow$ renewal, replacement, or retirement.

Magic is the motivating phenomenon. Causal compression is the mechanism. Queue migration and adaptive headroom are the systems consequences that make the mechanism testable beyond the local experience of invocation.

This paper is a conceptual framework and measurement program. Its equations define quantities, identify accounting identities, or specify bounded hypotheses; the framework’s claims are therefore evaluated relative to declared contracts, substrates, and measurement protocols.

2 The mechanism: from causal work to cheap invocation

Let a history be an element $h \in \mathcal{H}$, and let q denote a declared task contract (called a continuation contract below): the future decisions, observations, costs, risks, and outcomes that count for the analysis. A system may transform history into a present representation:

$$z_t = \phi(h_t).$$

The relevant question is not whether ϕ preserves every fact about the past. It is whether it preserves the distinctions required by q , while reducing the burden of future action.

Definition 1 (Representational causal compression). *A representation $\phi : \mathcal{H} \rightarrow \mathcal{Z}$ is representationally causally compressive for contract q when histories that map to the same present state are equivalent for the continuation problem under q , and the representation reduces the declared runtime burden of future decisions.*

A strict continuation-sufficiency condition is:

$$\phi(h_1) = \phi(h_2) \Rightarrow h_1 \sim_q h_2.$$

Here $\sim_q$ is an explicitly contract-relative equivalence relation: two histories are equivalent when they induce the same decision-relevant continuation behavior under the declared observations, costs, risks, and outcomes of q . The relation may mean equality of a conditional distribution, equality of policy value, or equality of reachable decision-relevant futures, depending on the contract; it is not assumed to be a global isomorphism. In practice, the representation may be a belief state, a learned parameter, a policy, a digest, a protocol position, or a maintained organizational routine.

Definition 2 (Structural causal compression). *A maintained structure is structurally causally compressive for transition e when historically distributed work is embodied in a persistent arrangement that makes e available at lower local invocation burden, conditional on the structure’s validity, maintenance, and continuation contract.*

Structural compression is not merely a shorter description. An airport network, a software library, or a service API changes which transitions are cheaply available. If z_i and z_j are states, an extensive capability can create an edge:

$$(z_i, z_j) \notin E_t \rightarrow (z_i, z_j) \in E_{t+1}.$$

Intensive progress instead improves an existing edge by lowering cost, latency, risk, or burden.

The two forms are coupled but not identical. A hitter compresses a previous at-bat into altered pitch recognition: primarily representational compression. An airline network embodies decades of engineering, regulation, scheduling, maintenance, and coordination: primarily structural compression. A contemporary AI system contains both a learned internal representation and a large external structure of data, chips, libraries, evaluation procedures, operators, and interfaces.

The coupling can be stated without pretending that all systems share one update rule:

$$\text{history} \rightarrow \begin{cases} \text{future-sufficient representation,} \\ \text{maintained transition structure,} \end{cases} \rightarrow \text{cheap invocation.}$$

The causal history is not annihilated. It is relocated into a state or structure whose local use is cheaper than its original construction.

2.1 The invocation–construction gap

For a transition e , substrate S , and continuation contract q , let $W_{\text{construction}}(e \mid S, q) > 0$ be the declared work added on top of inherited substrate S to make the transition reliable under q , and let $W_{\text{invocation}}(e \mid S, q) > 0$ be the work required for one local use. The substrate boundary is part of the measurement contract: it may include an existing model, chip, airport network, organizational routine, or other inherited capability, but it must be stated before comparison. This prevents an analysis of an AI service from silently expanding to metallurgy, electromagnetism, or human language. The same transition can therefore be analyzed at multiple nested boundaries; causal compression is scale-relative rather than boundary-free. Define the causal compression ratio:

$$G_e(S, q) = \frac{W_{\text{construction}}(e \mid S, q)}{W_{\text{invocation}}(e \mid S, q)}, \quad L_e(S, q) = \log(1 + G_e(S, q)).$$

The logarithmic form is useful when scales differ by orders of magnitude. Neither quantity is a literal magic score. They measure the asymmetry that can contribute to magical appearance.

A large $G_e(S, q)$ is not sufficient. A confusing bureaucracy may have a large construction–invocation gap while failing its contract. A pencil has low invocation cost but little hidden causal depth. The ratio measures asymmetry between construction and invocation, not amortization efficiency or whether the construction cost is justified by lifetime use. The phenomenology becomes plausible when the gap is large, the transition is reliable, its consequence is substantial, and the causal depth is not present in the local control loop.

Definition 3 (Magic-like transition). *A transition exhibits the phenomenology of technological magic under substrate S and contract q when it combines a high construction–invocation asymmetry $G_e(S, q)$, reliability at least sufficient for q , a consequential state change under the declared outcome measure, and low local visibility of the causal depth that supports the transition. This is a predicate over a declared observation protocol, not a scalar measure of subjective awe.*

3 Episodes, expertise, and structural memory

An episode may contain observations, actions, outcomes, and context:

$$e_t = (o_{0:t}, a_{0:t-1}, y_{0:t}, c_t).$$

An adaptive system need not carry the full episode into every future decision. It may evaluate the episode, alter a representation or structure, and retire the historical payload from active control:

$$\text{episode} \rightarrow \text{evaluation} \rightarrow \text{compiled change} \rightarrow \text{episode retirement}.$$

This gives a useful distinction between informational persistence and attentional persistence. Informational persistence is the survival of a lesson in a current representation or structure. Attentional persistence is the continued presence of the original episode in active control. Good adaptation often requires the first and suppresses the second.

A hitter's previous at-bat contains pitch sequence, timing, result, emotion, and context. The next pitch should not be controlled by replaying that entire record. Evaluation may alter pitch recognition, timing, or a release-point cue. The narrative payload can be retired while its decision-relevant consequence remains.

History has become morphology.

The same sequence appears in technical systems. A one-time repair becomes a test. A repeated test becomes a deployment gate. Repeated deployments become a service. A service becomes infrastructure with operators, documentation, monitoring, dependencies, and replacement paths. Expertise is history compiled into cheap present action.

Compilation is therefore not disappearance. It is a transfer of burden from the invocation loop into a representation or structure that must remain valid. Structural memory is history compiled into continuation-relevant present structure; it does not require retaining an event log.

4 A lifecycle of compressed transitions

Represent an adaptive system as an evolving attributed transition network:

$$\mathcal{N}_t = (Z_t, E_t, A_t, \Lambda_t, M_t),$$

where Z_t are states, E_t available transitions, A_t transition attributes, Λ_t realized flows, and M_t maintenance state. An edge may carry capability, invocation cost, latency, reliability, evidence quality, and capacity:

$$e_{ij} = (k_{ij}, c_{ij}, \tau_{ij}, \rho_{ij}, v_{ij}, \kappa_{ij}).$$

The causal-compression lifecycle is:

$$\begin{aligned} &\text{generation} \rightarrow \text{evaluation} \rightarrow \text{selection} \rightarrow \text{compilation} \\ &\rightarrow \text{maintenance} \rightarrow \text{re-evaluation} \rightarrow \{\text{renew, replace, retire}\}. \end{aligned}$$

Generation creates candidates. Evaluation determines whether a candidate satisfies a declared contract. Selection turns a possibility into a commitment. Compilation moves causal work out of the immediate invocation loop. Maintenance preserves the validity of the compiled transition. Re-evaluation compares the incumbent with currently reachable substitutes. Renewal is recommitment

after comparison; it is not automatic privilege for the incumbent. Replacement changes the maintained implementation. Retirement attempts to release obligations without destroying required continuation capability.

A commercial technology may instantiate a narrower lifecycle:

idea $\rightarrow$ demo $\rightarrow$ adoption $\rightarrow$ market $\rightarrow$ infrastructure,

but that is one domain-specific projection, not the general theory.

5 Worked system: software as compressed causal work

Consider a recurring problem in a software organization.

1. A developer manually solves the problem once.
2. The solution is written into documentation.
3. The documentation becomes a script.
4. The script becomes a library.
5. The library becomes a service.
6. The service becomes infrastructure used by other teams and external users.

This is a stylized progression, not a claim that every script becomes a library or every service becomes infrastructure. At each step, invocation becomes cheaper for the next user because earlier work has been compiled into structure. The reachable-state graph expands: tasks that previously required bespoke reasoning become available through a stable edge. But the causal work has not been eliminated. It has changed location.

The library now requires tests, dependencies, maintainers, compatibility guarantees, security updates, deployment procedures, observability, documentation, and an exit path. The service may be locally cheap to invoke while globally expensive to preserve. A successful compression therefore creates both capability and commitment.

Suppose the service is eventually obsolete. Deleting its source files does not by itself recover the capacity previously consumed by the service. Migration, retraining, compatibility closure, residual dependencies, contractual closure, and temporary parallel operation determine whether retirement releases or consumes future capacity. This is why nominal deletion and capacity-releasing retirement are different events.

The example also exposes the downstream bottleneck. If candidate generation becomes cheap while evaluation, integration, maintenance, and retirement remain fixed, the organization does not obtain unlimited realized capability. The queue migrates:

generation $\rightarrow$ evaluation $\rightarrow$ integration $\rightarrow$ maintenance $\rightarrow$ retirement.

Cheap generation can therefore increase the stock of unprocessed commitments faster than the system can safely absorb them. This queue-migration prediction is the most direct empirical entry point in the framework: it can be tested without estimating the total causal history of the system, by varying upstream candidate-generation capacity while holding downstream service capacities fixed and measuring where backlog and delay accumulate.

6 Experimental probes of queue migration

The queue-migration claim is tested in two deliberately different evidence layers. The synthetic intervention tests whether the instrument can recover a known bottleneck under controlled changes. The trace layer tests workload-shape sensitivity using one trace from an archive corpus.

6.1 Instrument and intervention

The frozen v3 protocol separates two questions. The capacity-release ladder is a positive control: when one lifecycle stage is deliberately constrained, does the instrument identify that known stage? The generation sweep is the queue-migration probe: when upstream generation pressure rises while downstream capacity remains fixed, does the first visible bottleneck move downstream as predicted?

6.2 Synthetic positive controls

The capacity-release ladder uses a finite, stage-ordered service system with fixed downstream capacities and constrains exactly one of evaluation, integration, maintenance, or retirement in turn. Each record reports arrivals, completions, stage flow, queue state, and drain time. The independent auditor checks generated = arrival total = completed, stage-flow conservation, empty final queues, and finite drain completion. The ladder localized evaluation, integration, maintenance, and retirement respectively across all three seeds and both workload families. This is a positive-control result: it establishes that the instrument can recover a declared bottleneck.

6.3 Generation sweep

The same protocol uses two independently implemented finite workload families: five-tick burst/on-off arrivals ('burst_train') and bounded seeded jitter around the declared mean ('jittered'). Each workload runs for 80 ticks and must drain completely. The generation sweep uses rates 1, 2, 4, 8 with downstream capacities fixed at 4. Seeds 11, 12, and 13 are applied to both workload families, producing 48 records.

The generation sweep is the more substantive queue-migration probe. Rates 1, 2, and 4 remain generation-bound; rate 8 becomes evaluation-bound. The result is not merely that a deliberately constrained stage queues: increasing upstream generation pressure exposes the first constrained downstream stage under the frozen dynamics. All 48 records pass flow conservation and drain within the finite bound. Figure 1 renders both the positive-control ladder and the generation-sweep transition. The strongest interpretation is instrument identifiability under the declared synthetic contract. This validates the instrument, not the general theory.

For reproduction, the protocol, input hash, seeds, condition definitions, independent auditor, and result receipt are preserved in the experiment repository. The executable checks are:

```
PYTHONPATH=src python3 -m unittest tests.test_queue_migration_v3_bursty -v
PYTHONPATH=src python3 scripts/run_queue_migration_v3_bursty.py \
  --output artifacts/queue_migration_v3.json \
  --summary artifacts/queue_migration_v3.md
```

The finite drain requirement and independent flow-conservation audit are hard gates rather than post-hoc diagnostics.

Bottleneck localization in the frozen synthetic v3 protocol

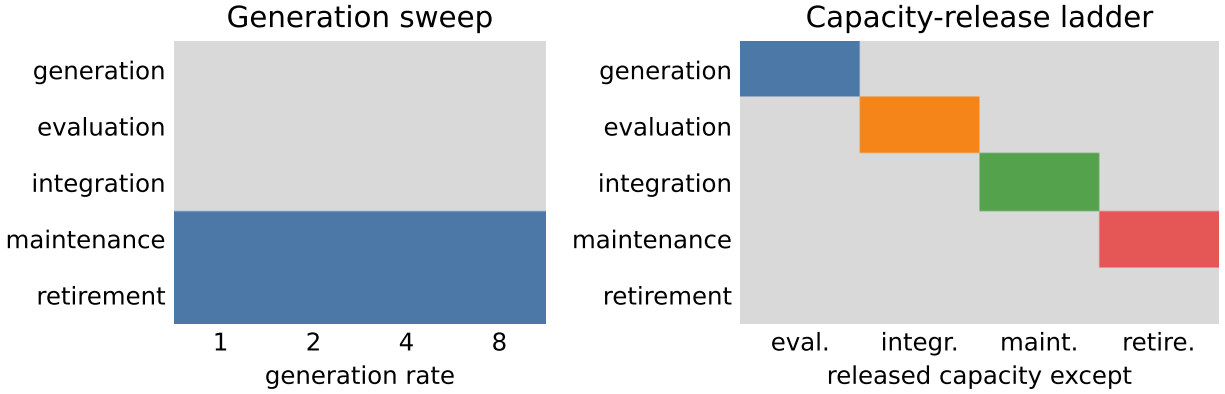


Figure 1: Bottleneck localization in the frozen synthetic v3 protocol. In the generation sweep, the observed bottleneck moves from generation to evaluation only at the highest rate. In the positive-control ladder, the localized bottleneck follows the constrained lifecycle stage. The diagram summarizes 48 audited records, not a continuous estimate over untested rates.

6.4 Trace-derived arrival calibration

One workload trace from a single archive corpus replays 1,190 timestamped records as anonymized one-minute demand bins, using only timestamps and role labels. The frozen trace protocol fixes the source hash, excludes message content, and repeats the same generation sweep and capacity-release ladder without retuning. The trace tests robustness to an empirically observed arrival-process shape; it does not establish that the source archive itself implements the proposed generation/evaluation/integration/maintenance/retirement lifecycle. It also does not establish stationarity, participant causality, or deployment generality.

6.5 Evidence boundary

The three layers therefore have different estimands: the positive controls test localization against known constraints; the generation sweep tests bottleneck migration under controlled upstream pressure; and the trace replay tests sensitivity to a non-synthetic arrival process. Causal compression does not abolish scarcity. By reducing burden at one stage, it can expose scarcity at the next: cheaper generation can make evaluation, verification, integration, maintenance, or retirement the visible constraint.

7 Commitment, maintenance, and adaptive headroom

Separate raw capability from the capacity to change:

$$\mathcal{C}_t = \text{raw capabilities possessed,}$$

$$K_t = \text{gross capacity to deploy, coordinate, modify, replace, and support change,}$$

$$M_t = \text{capacity encumbered by current commitments.}$$

The scalar headroom approximation is:

$$H_t = K_t - M_t.$$

Headroom is not the value of the future. The richer object is the reachable future set:

$$\mathcal{O}_t = \{z : z \text{ remains reachable within acceptable time, cost, risk, and commitment bounds}\}.$$

The scalar is a diagnostic proxy for the option set, not a substitute for it.

Selection creates commitment. Commitment encumbers some adaptive capacity, even when the resulting capability expands the reachable future set. Thus $\mathcal{O}_t$ can expand while H_t contracts. Maintenance keeps that commitment alive. A rough maintenance balance is:

$$\dot{M} = r_{\text{compile}}\bar{m}_{\text{new}} - r_{\text{retire}}\bar{m}_{\text{released}} + D_M,$$

where D_M includes dependency aging, standards changes, expertise loss, security requirements, and other drift. The equation is an accounting model; it becomes informative only when its terms are operationally measured.

The capability/headroom distinction is therefore immediate but not universal in its empirical interpretation:

$$|\mathcal{C}_t| \uparrow \not\Rightarrow K_t \uparrow \not\Rightarrow H_t \uparrow.$$

A system can acquire capabilities while losing the capacity to modify, replace, or retire them.

7.1 Safe retirement and headroom-positive retirement

Safety and headroom are related but not identical. Let a retirement action preserve a required continuation, reliability, or risk contract. That preservation condition is the safety condition. Separately, define the net headroom change:

$$\Delta H_e^{\text{retire}} = \Delta K_e - \Delta M_e.$$

A retirement is *headroom-positive* when:

$$\Delta H_e^{\text{retire}} > 0.$$

A safe retirement may be temporarily headroom-negative, for example during a security migration or mandated parallel run. Conversely, a headroom-positive deletion can be unsafe if it destroys required continuation capability. The two predicates must not be collapsed.

Behavioral persistence is not automatically irrationality. A persistent incumbent is maladaptive only relative to a counterfactual policy with the same information and declared objective. Replacement cost, transition risk, and temporary duplication belong in that comparison.

8 The phenomenology of magic

Causal compression creates an asymmetry between construction and invocation. The first successful flight required a new arrangement of knowledge, materials, engines, control systems, pilots, airports, regulation, fuel, maintenance, and coordination. A mature airline system makes one passenger's traversal feel ordinary.

Cheap traversal is often evidence of expensive accumulated structure.

A prompt can produce a competent-looking result because models, data, chips, libraries, interfaces, evaluation procedures, operators, and global infrastructure have already absorbed much of the causal work. The local invocation is cheap relative to the historical work that made it possible. This is the return of magic: not supernatural causation, but the experience that consequential transformations arrive through opaque, low-latency interfaces.

The distinction between construction and invocation also clarifies accountability. A low local burden does not remove responsibility for provenance, maintenance, failure modes, operators, or reversal paths. A reliable transition can be invoked without the user understanding its mechanism; an apparently transparent explanation can still be false. Inspectability should therefore be targeted where it changes decisions: evidence, provenance, known failure modes, maintenance state, and exit paths. Total causal visibility is neither possible nor necessary.

9 Predictions and measurement program

The framework makes bounded predictions. These are not consequences of the definitions alone.

1. **Infrastructure signature.** Transitions that become durable infrastructure should exhibit persistent reductions in invocation burden without equivalent loss of contract performance, conditional on continued maintenance and a stable contract.
2. **Compression–burden relocation.** Increasing causal compression should decrease the fraction of lifecycle work performed during invocation and increase the fraction borne by pre-existing representation, structure, and maintenance. In a declared workload accounting, this predicts $W_{\text{invoke}}/W_{\text{lifecycle}} \downarrow$, not necessarily $W_{\text{maintain}} \uparrow$; consolidation can reduce total work.
3. **Queue migration.** Increasing candidate-generation capacity should move observed bottlenecks downstream when evaluation, integration, maintenance, or retirement capacity does not scale with it.
4. **Headroom erosion.** Holding nominal capability performance constant, faster growth in commitment burden and maintenance drift should narrow measured adaptation choices.
5. **Support-architecture sensitivity.** Transitions whose reliable invocation depends on a larger unresolved support set should be more sensitive to support interruption, conditional on the availability of substitutes. This tests dependency architecture, not compression magnitude alone: compression and fragility are orthogonal properties.
6. **Retirement asymmetry.** Deleted-edge counts should overestimate recovered capacity when migration, replacement, residual risk, or capability loss are not charged.
7. **Timescale matching.** Under environmental change, an intermediate structural-update regime should outperform both excessively rapid updating and excessively slow updating on a predeclared adaptive-performance measure. The U-shape is a hypothesis requiring a specified dynamical model and repeated seeds.

A minimal instrument should log, for each candidate transition: its contract, evidence status, construction workload, invocation workload, evaluation cost, promotion decision, compilation cost, maintenance burden, re-evaluation outcome, retirement cost, capability loss, and net headroom change. It should distinguish the capability lifecycle:

candidate $\rightarrow$ demonstrated $\rightarrow$ repeatable $\rightarrow$ discoverable
 $\rightarrow$ routable $\rightarrow$ trusted $\rightarrow$ economical $\rightarrow$ maintained.

Synthetic fixtures can test instrumentation and implementation invariants. They cannot establish market, institutional, civilizational, or general AI effects. A serious test program should compare generation, evaluation, integration, maintenance, and retirement capacity under controlled workload scaling, with fixed seeds, independent correctness checks, and explicit receipts. For queue-migration tests, arrival rate, service rate, stage occupancy, and time-in-stage should be reported separately; Little’s law is a useful benchmark only when its stationarity and finite-mean conditions are checked, not a license to infer permanent overload from a transient spike [10]. For update-rate tests, adaptation and retention should be measured as separate outcomes: a policy that changes quickly may gain short-run fit while losing previously retained behavior, the stability–plasticity tradeoff documented in connectionist learning [7].

10 Relation to adjacent frameworks

The contribution claimed here is unification, not ownership of every component. Representational compression overlaps with state abstraction, sufficient statistics, learned representations, and causal compression in existing technical senses [1, 9, 16]. Structural compression overlaps with compilation, amortization, infrastructure studies, organizational routines, and software evolution [8, 13, 14]. Commitment and headroom connect to technical debt, path dependence, lock-in, maintenance engineering, and real-options reasoning [2, 5, 6]. Queue migration is adjacent to queueing theory, but the paper’s stage model is not Little’s law and should not inherit its assumptions silently [10]. The update-rate hypothesis is adjacent to the stability–plasticity problem in continual learning [7], while remaining a broader systems hypothesis rather than a neural-network claim. The treatment of magical appearance also relates to scholarship on how technological systems are staged and made opaque [4].

These literatures contribute different mechanisms rather than interchangeable synonyms. State-abstraction work asks which distinctions can be discarded while preserving planning or learning properties. Amortized analysis moves cost across a sequence of operations rather than treating each invocation in isolation. Organizational-memory and routine theories locate retained knowledge in distributed procedures and roles [11, 15]; infrastructure studies show that supposedly transparent access depends on standards, maintenance, and situated work. Simon’s account of hierarchy explains why stable intermediate structures can make complex systems tractable [12], while Clark and Chalmers make a related argument about cognition distributed across external supports [3]. Arthur’s increasing-returns model shows how selection can make a path difficult to reverse, and real-options theory explains why waiting or preserving alternatives can have value under uncertainty. The present framework does not collapse these results into one theory; it proposes a lifecycle relation among them and supplies a shared measurement vocabulary for construction, invocation, commitment, maintenance, and retirement.

The proposed synthesis is the lifecycle relation among these objects: historical causal work becomes a cheap transition; cheap transitions expand reachable states; selection turns options into commitments; maintenance preserves the compressed structure; and retirement determines whether future optionality is recovered. The novelty claim is therefore a framework-level one and remains subject to comparison with adjacent literatures.

11 Limitations and non-claims

Causal compression is not causal understanding: hidden causality must be distinguished from absent causality, and reliability still requires evidence. The scalar headroom model is an approximation;

real systems have shared infrastructure, complementarities, bottlenecks, nonconvex migration costs, dependencies, and vector-valued constraints, so $\mathcal{O}_t$ is the more faithful target.

Construction work, invocation work, maintenance burden, and capacity need not be commensurable without a declared contract and valuation rule. The gap metric and lifecycle equations are therefore instrument-relative accounting objects. A temporary arrival rate above service capacity need not imply permanent overload if slack or later service clears the queue.

The framework does not claim that every successful compression improves welfare, that every persistent structure is irrational, or that retirement is always beneficial. This paper contains no newly collected field dataset, no energy measurement, and no empirical estimate of AI’s social or economic effects. Its operational observation uses existing public traces rather than a purpose-built field study; the predictions are proposed instruments and bounded observations, not general organizational or AI-system effects.

12 Conclusion

Civilization becomes more powerful by moving causal work out of the present. Procedures become policies. Policies become infrastructure. Repeated flows become commitments. The causal chain remains real even when the invocation becomes small.

This is why the title is not merely a metaphor. The invocation–construction gap supplies one measurable component of magical appearance, while the magic-like predicate adds reliability, consequence, and local causal visibility. A transition feels magical when it is reliable and consequential, cheap to invoke, expensive to construct within a declared substrate boundary, and causally deep beyond the local interface.

Compression converts past effort into present capability. Commitment converts present capability into future obligation. The past survives as structure, and structure has to be maintained. Retirement is the mechanism that can break the obligation while preserving required continuation capability and restoring future degrees of freedom.

The central lifecycle is therefore:

learn → create options → select commitments → compile capability
→ maintain → re-evaluate → renew, replace, or retire → recover option value.

The shortest statement of the framework is:

Causal compression makes the past cheap to invoke;
adaptive headroom determines whether the future remains changeable.

Or, less technically:

Learning creates options. Selection creates commitments.
Maintenance creates memory. Retirement preserves possibility.

References

- [1] Pierre-Olivier Amblard et al. Causal compression. *arXiv preprint arXiv:1611.00261*, 2016. URL <https://arxiv.org/abs/1611.00261>. Source checked via ar5iv rendering; bibliographic author list should be completed before submission.

- [2] W. Brian Arthur. Competing technologies, increasing returns, and lock-in by historical events. *The Economic Journal*, 99(394):116–131, 1989. doi: 10.2307/2234208. URL <https://doi.org/10.2307/2234208>.
- [3] Andy Clark and David J. Chalmers. The extended mind. *Analysis*, 58(1):7–19, 1998. doi: 10.1093/analys/58.1.7. URL <https://doi.org/10.1093/analys/58.1.7>.
- [4] Kelley Cotter et al. Conjuring algorithms: Understanding the tech industry as stage magicians. *Social Media + Society*, 2024. doi: 10.1177/14614448241251789. URL <https://doi.org/10.1177/14614448241251789>. Source checked via publisher and eScholarship landing pages; author list should be completed before submission.
- [5] Ward Cunningham. The WyCash portfolio management system. In *Proceedings of the 1992 ACM Conference on Object-Oriented Programming Systems, Languages, and Applications*, 1992. URL <https://c2.com/doc/oops1a92.html>.
- [6] Avinash K. Dixit and Robert S. Pindyck. *Investment under Uncertainty*. Princeton University Press, 1994. ISBN 9780691034102. URL <https://press.princeton.edu/books/hardcover/9780691034102/investment-under-uncertainty>.
- [7] Robert M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999. doi: 10.1016/S1364-6613(99)01294-2. URL [https://doi.org/10.1016/S1364-6613\(99\)01294-2](https://doi.org/10.1016/S1364-6613(99)01294-2).
- [8] Meir M. Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076, 1980. doi: 10.1109/PROC.1980.11805. URL <https://gvern.net/doc/cs/1980-lehman.pdf>.
- [9] Lihong Li, Thomas J. Walsh, and Michael L. Littman. Towards a unified theory of state abstraction for MDPs. In *Proceedings of the Ninth International Symposium on Artificial Intelligence and Mathematics*, 2006. URL <http://rbr.cs.umass.edu/aimath06/proceedings/P21.pdf>.
- [10] John D. C. Little. A proof for the queuing formula: $L = \lambda w$. *Operations Research*, 9(3):383–387, 1961. doi: 10.1287/opre.9.3.383. URL <https://doi.org/10.1287/opre.9.3.383>.
- [11] Richard R. Nelson and Sidney G. Winter. *An Evolutionary Theory of Economic Change*. Harvard University Press, 1982. ISBN 9780674272286. URL <https://www.hup.harvard.edu/books/9780674272286>.
- [12] Herbert A. Simon. The architecture of complexity. *Proceedings of the American Philosophical Society*, 106(6):467–482, 1962. URL https://sites.cc.gatech.edu/classes/AY2013/cs7601_spring/papers/Simon-Complexity.pdf.
- [13] Susan Leigh Star and Karen Ruhleder. Steps toward an ecology of infrastructure: Design and access for large information spaces. *Information Systems Research*, 7(1):111–134, 1996. doi: 10.1287/isre.7.1.111. URL <https://doi.org/10.1287/isre.7.1.111>.
- [14] Robert Endre Tarjan. Amortized computational complexity. *SIAM Journal on Algebraic and Discrete Methods*, 6(2):306–318, 1985. doi: 10.1137/0606031. URL <https://doi.org/10.1137/0606031>.

- [15] James P. Walsh and Gerardo R. Ungson. Organizational memory. *Academy of Management Review*, 16(1):57–91, 1991. doi: 10.5465/amr.1991.4278992. URL <https://doi.org/10.5465/amr.1991.4278992>.
- [16] Liang Wendong, Simon Buchholz, and Bernhard Schölkopf. Algorithmic causal structure emerging through compression. In *Proceedings of the Fourth Conference on Causal Learning and Reasoning*, volume 275, pages 201–242. Proceedings of Machine Learning Research, 2025. URL <https://proceedings.mlr.press/v275/wendong25a.html>.